UNIVERSIDAD NACIONAL DE LA PATAGONIA AUSTRAL
Unidad Académica Río Gallegos

Estructura de Datos

Tema 7:
ARBOL B

Docente: Mg. Héctor Reinaga

Índice

- Introducción
  - Árbol binario
  - Árbol multicamino
    - Concepto
    - Ventajas y desventajas
- Árbol B
  - Concepto
  - Propiedades
  - Aplicaciones
  - Estructura de nodos
  - Inserción: algoritmo, ejemplo
  - Búsqueda
  - Borrado: proceso, ejemplo
- Tipos de Árbol B
  - Árbol B*
  - Árbol B+

Introducción

- Árbol Binario:
  - Tiene como máximo dos descendientes
- Árbol multicamino
  - Tiene más de un descendiente (hijo)
  - Se representan en memoria de las siguientes maneras:
    - Mediante arreglos
    - Mediante lista de hijos
    - Basada en árboles binarios

Introducción

- Motivación:
  - Las variantes de árboles binarios fueron desarrollados para funcionar en memoria principal.
  - Volumen de información:
    - Hay aplicaciones en las que el volumen de la información que no caben en la memoria principal y es necesario almacenarlos en dispositivos de almacenamiento secundario.
    - Esta organización debe adecuada para recuperar los datos en forma eficiente.

Introducción

Ejemplo:

Supongamos que tenemos un millón de datos en memoria secundaria (por ejemplo disco rígido)

- Árbol binario
  - Tiempo promedio para localizar alguno de los nodos es de $\log_2 n$ (nodos del árbol) accesos a disco.
  - Si el árbol contiene 1.000.000 de elementos. Se necesitarían aproximadamente **20 accesos** a disco.
- Árbol B
  - Cuyo orden en este caso es de 100 (página con 100 claves).
  - Se necesitan como máximo **3 accesos** a disco. (Log_{100} 1.000.000).
  - Los accesos disminuyen de forma considerable (**3 en vez de 20**).

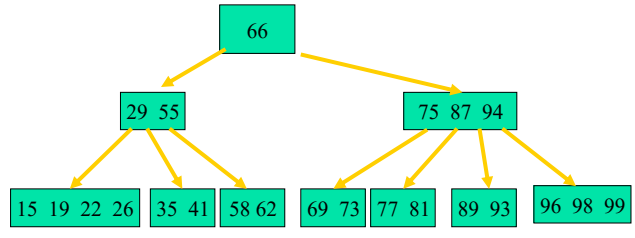
Árbol B: concepto

- Creados en 1970 por Rudolf Bayer y Ed McCreight
  - Hoy en día son el estándar para la indexación de archivo de datos.
- Concepto
  - Es un árbol de búsqueda que puede estar vacío o aquel cuyos nodos pueden tener varios hijos, existiendo una relación de orden entre ellos.
- Son una generalización de los árboles balanceados.
  - Requiere que todos los nodos hoja se encuentren a la misma altura.
- Como trabaja?
  - El Árbol B accede al archivo mediante bloques de bytes
  - Esto reduce el número de accesos al dispositivo de almacenamiento secundario.
- Representan una *búsqueda externa*, utilizando *árboles multicaminos.*
- Su principal característica es su buen rendimiento en cuanto a operaciones de búsqueda, aunque veremos que esto supone un coste en las operaciones de inserción y borrado.

Árbol B: propiedades

- Un árbol-B de orden **n** es un árbol que satisface las siguientes propiedades:
  1. Cada nodo tiene como máximo **n** hijos.
  2. Cada nodo (excepto raíz y hojas) tiene como mínimo n/2 hijos.
  3. La raíz tiene al menos 2 hijos, sino es un nodo hoja.
  4. Todos los nodos hoja aparecen al mismo nivel y el camino de la raíz a las hojas tiene la misma longitud.
  5. Un nodo no hoja con k hijos contiene k-1 elementos almacenados (claves).
  6. Los hijos que cuelgan de la raíz (r_1, ⋯, r_m) tienen que cumplir ciertas condiciones:
     1. El primero tiene valor menor que r_1.
     2. El segundo tiene valor mayor que r_1 y menor que r_2, etc.
     3. El último hijo tiene valor mayor que r_m.
  7. Dentro de cada hoja las claves se encuentran ordenadas en forma creciente.

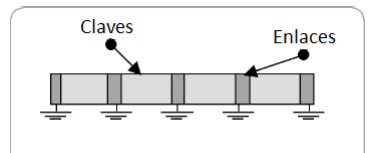
Árbol B: ejemplo



- Orden del árbol: 5
- Altura del árbol: 3
- Claves: números enteros
- Todas las páginas contienen: 2, 3 o 4 elementos, excepto la raíz
- Todas las hojas están al mismo nivel
- Claves: ordenación izquierda a derecha dentro de cada página. Dividen a los nodos descendientes a la manera de un árbol de búsqueda:
  - claves nodo izquierdo menores y claves de nodo derecho mayores.

Árbol B: estructura de nodos

- Representación de nodo (página):



Nodo para un árbol B de orden 5

Árbol B: almacenamiento

- A continuación se observa un gráfico que ejemplifica como se almacenan los datos en un Árbol B:
  - Se almacenan hacia la izquierda los datos menores al elemento en el nodo padre.
  - Hacia la derecha los mayores al mismo, lo que genera un método de ordenación.

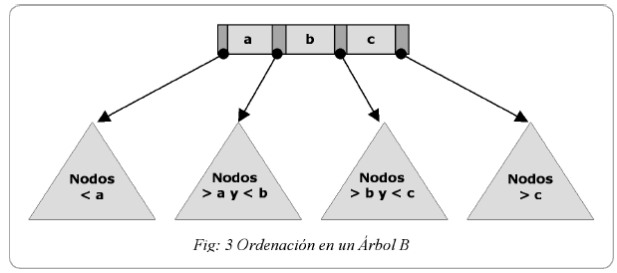
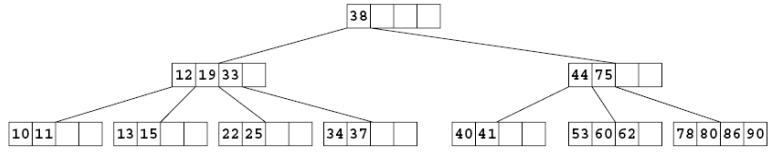


Fig: 3 Ordenación en un Árbol B

Árbol B: estructura de nodos

- Dado el siguiente árbol B:



- Cada elemento (clave) de un nodo interno actúa como un valor separador, que lo divide en subárboles.
  - Por ejemplo, si un nodo interno tiene tres nodos hijos, debe tener dos valores separadores o elementos *a1* y *a2*.
  - Todos los valores del subárbol izquierdo deben ser menores a *a1*, todos los valores del subárbol del centro deben estar entre *a1* y *a2*, y todos los valores del subárbol derecho deben ser mayores a *a2*.
- Para todos los nodos internos, exceptuando la raíz, el número de elementos es uno menos que el número de punteros a nodos (hijos).

Árbol B: estructura de nodos (continuación)

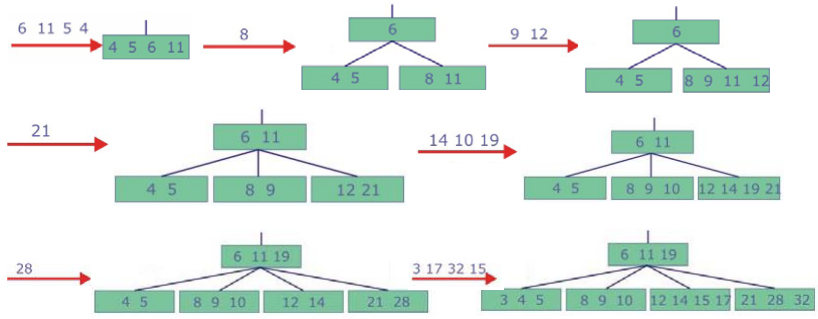
- Nodo lleno:
  - Nº de claves > n-1 (n es el orden del árbol)
- Dos nodos que están a medio llenar pueden juntarse para formar un nodo.
- Un nodo lleno puede dividirse en dos nodos (si es que hay lugar para subir un elemento al nodo padre).
- Estas propiedades hacen posible que el árbol B se ajuste para preservar sus propiedades ante la **inserción** y **eliminación** de elementos.
- Los nodos hoja tienen la misma restricción sobre el número de elementos, pero no tienen hijos.

Pregunta:

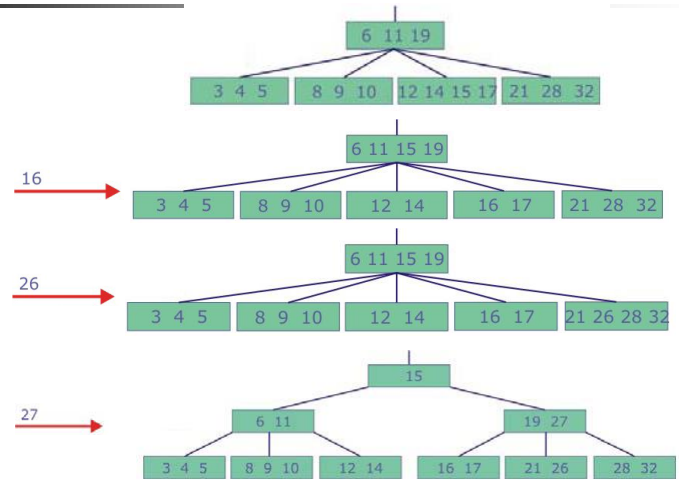
- Como crecen los árboles?
  - Hacia arriba
    - Nodo hojas al mismo nivel
    - La mediana se promociona hacia arriba

Árbol B: inserción

- Ejemplo:
  - Creación de árbol B de orden 5 insertando claves de enteros:
    - 6 - 11 - 5 - 4 - 8 - 9 - 12 - 21 - 14 - 10 - 19 - 28 - 3 - 17 - 32 - 15 - 16 - 26 - 27



Árbol B: inserción



Búsqueda en árboles B

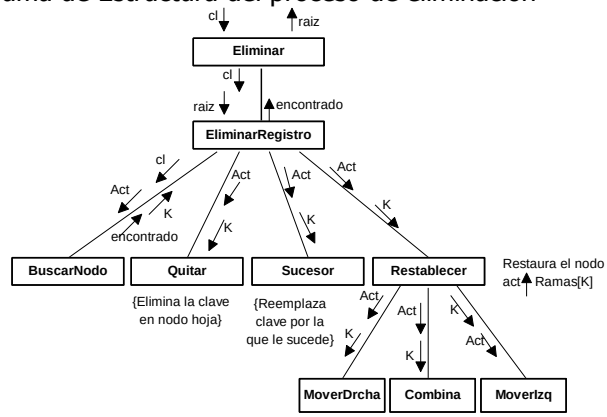
- Misma estrategia que para árboles binarios de búsqueda.
- Pero cuando estamos posicionados en una página (nodo) hay que inspeccionar las claves de que consta.
  - Resultado inspección: posición de la clave o camino a seguir en la búsqueda.
- Procedimiento **buscar()**:
  - Entrada=raíz del árbol B y clave a localizar
  - Salida= valor lógico (TRUE si encuentra clave), la dirección de la página que contiene la clave y la posición dentro del nodo.
  - El examen de cada página se realiza en un procedimiento auxiliar **buscarNodo()**.

Eliminación

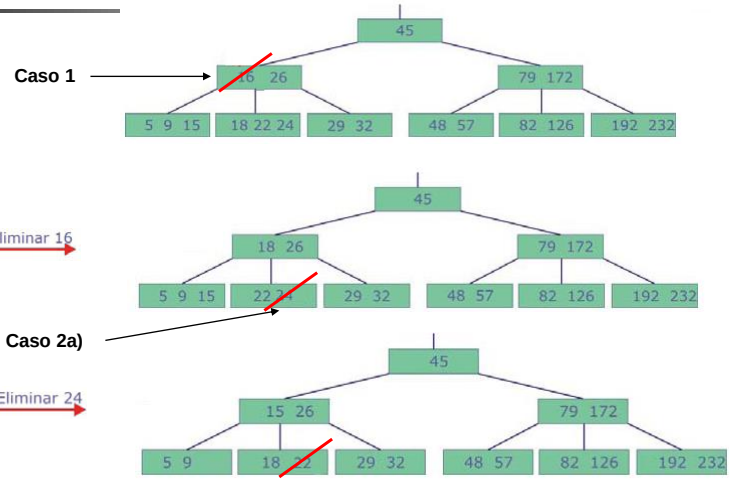
- Misma forma que las inserciones
  - Se insertan/eliminan claves en una página hoja.
- Premisa:
  - Si una clave no está en una hoja, su clave antecesora o sucesora en orden natural estará en una hoja.
- *Caso 1: la hoja donde se va a eliminar tiene nº claves>mínimo*
  - Si la clave a eliminar no está en una hoja, se pondrá a la clave predecesora en la posición ocupada por la clave eliminada y se suprimirá la clave en la hoja.
- *Caso 2: Si la hoja donde se va a eliminar tiene nº claves=mínimo*
  - Realizar movimiento de claves para que siga siendo árbol B.
  - Examinamos hojas adyacentes al nodo con mismo antecedente.
  - Caso 2a: Si una de las hojas tiene nº claves > mínimo
    - Subir una clave al nodo padre para que a su vez descienda de éste, otra clave al nodo que se quiere restaurar.
  - Caso 2b: Si hojas contiguas tienen nº claves = mínimo
    - Elegir la hoja a eliminar, su hoja anterior y la mediana de ambas, procedente del nodo antecedente, y se combinan como un nuevo nodo hoja. El proceso se puede propagar hacia arriba hasta suprimir la raíz y disminuir la altura del árbol B.

Eliminación

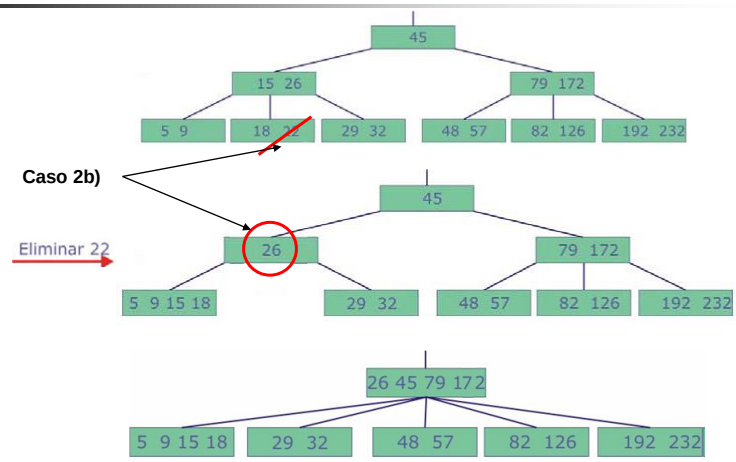
- Diagrama de Estructura del proceso de eliminación



Eliminación

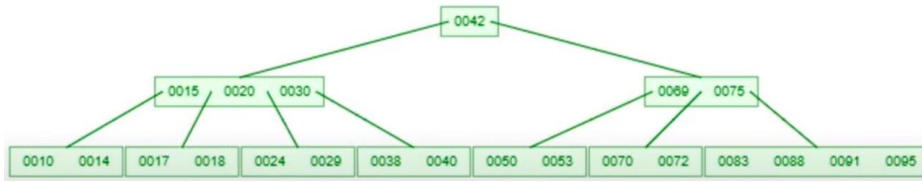


Eliminación

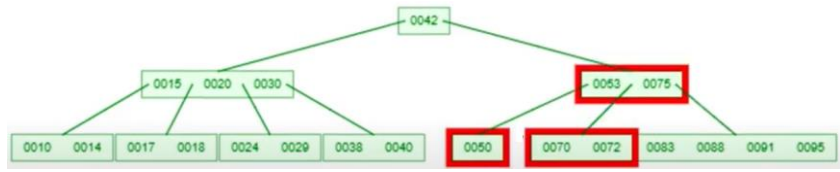


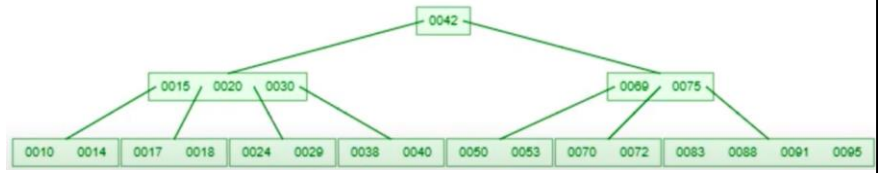
Otro caso

- Se insertaron 18, 24 y 29
- Se borra el nro. 69

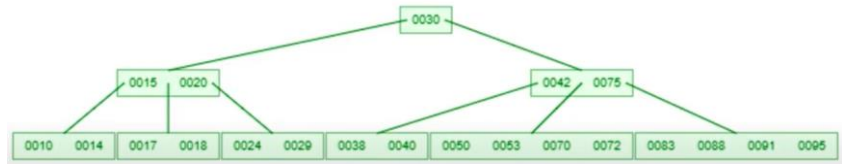


Otro caso...





Otro caso



Implementación Arbol B

Clase	Atributos	Métodos
BTree	BNode root int arity	BTree(int arity) void addKey(Object key) boolean searchKey(Object key) boolean deleteKey(Object key)
BNode	Keys data BNode[] children int arity, size	BNode(int arity) BNode(int arity, Keys data) void setData(Keys data) Keys getData() BNode getChild(int index) BNode addChild(BNode data) BNode addChild(BNode data, int index) BNode deleteChild(int index) void removeChild(BNode data) int degree()
Keys	int arity, size Object[] keys	Keys(int arity) Object keyAt(int index) int search(Object key) int addKey(Object key) boolean deleteKey(Object key)
Transversal	private Vector nodes	public Transversal(BTree AB) public boolean hasMoreElements() public Object nextElement()

Árbol B: aplicaciones

- Creación de bases de datos
  - El árbol B es una forma de implementar los índices de una base de datos relacional.
- Gestión del sistema de archivos del sistema operativo OS/2:
  - Para aumentar la eficacia en la búsqueda de archivos por los subdirectorios.
- Sistemas de compresión de datos
  - Árboles B para la búsqueda por clave de datos comprimidos.
- Se utilizan en búsquedas externas con el objeto de minimizar el número de accesos al disco.
  - Acceso a disco=>lectura de bloque de información.
  - Hasta ahora:
    - Árboles funcionando en memoria principal.
  - Si volumen de datos grande:
    - manejar los datos directamente sobre un dispositivo de almacenamiento externo.

Tipos de Árbol B

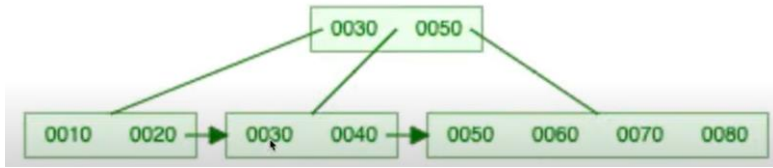
- Árboles B: datos en nodos internos y hojas
- Arboles B*
- Árboles B+(multicaminos)
  - Permiten recorrido secuencial rápido
  - Todas las claves se encuentran en hojas, duplicándose en la raíz y nodos interiores aquellas que resulten necesarias para definir los caminos de búsqueda.
  - Para facilitar el recorrido secuencial rápido las hojas se pueden vincular, obteniéndose una trayectoria secuencial para recorrer las claves del árbol.
  - Ocupan algo más de espacio que los árboles B, pues existe duplicidad de algunas claves.
  - Las claves de las páginas raíz e interiores se utilizan únicamente como índices.

Inserciones

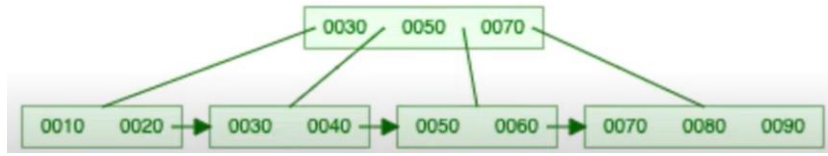
- Similar a Arbol B
- Sin underflow, es igual
- Con underflow, se divide en 2, pero duplicando (del lado de mayores)

Insercion: ejemplo

- Insertar 90



Inserción



Consultas…

¿?

Fin de la teoría EDA